

1.

Consider the multicycle E20 processor. Consult the circuit diagram in the E20 manual. Using only that diagram and the instruction reference, determine which E20 instruction will be performed with the following control line settings for each of the stages:

```
IF stage:
    WEir = 1
    WEram = 0
    Wereg = 0
    WEpc = 0

    WEa = 0
    WEB = 0
    WEalu = 0
    MUXaddr = 1
ID stage:
    WEa = 1
    WEB = 1
    MUXr1 = 4
    MUXb = 0
EXEC stage:
    aluOp = 0
    WEalu = 1
    MUXalu = 1
MEM stage:
    MUXaddr = 0
    WEram = 0
WB stage:
    WEpc = 1
    MUXpc = 0
```

2.

Consider the following configuration of control lines in a single-cycle E20 processor:

- $\text{FUNC}_{\text{alu}} = 2$
- $\text{MUX}_{\text{alu}} = 0$
- $\text{MUX}_{\text{pc}} = 3$
- $\text{MUX}_{\text{rf}} = 1$
- $\text{MUX}_{\text{tgt}} = 1$
- $\text{MUX}_{\text{dst}} = 2$
- $\text{WE}_{\text{rf}} = 1$
- $\text{WE}_{\text{dmem}} = 0$

Which E20 instruction does this configuration correspond to? Explain your answer by justifying the setting of each of the control lines.

3.

Now consider an E20 computer that has two caches:

- the L1 cache is an 8-row direct-mapped cache with a blocksize of 32 bits.
- the L2 cache is a 4-row 2-way set-associative cache with a blocksize of 64 bits.

E20 memory addresses are 13 bits. Memory cells are 16 bits. The caches are initially empty. Instruction reads are not cached (in other words, only `lw` and `sw` affect the cache).

Write an E20 assembly language program that will generate a memory reference that causes a miss on L1, but a hit on L2. In a comment in your program, specify the tag and index of the access in question. Your program should be as short as possible.

4.

Consider the following Translation Lookaside Buffer (TLB). It's a fully-associative cache with a size of 8 entries. The page size is 4 cells. Each memory cell is one byte.

Virtual page	V	Dirty	Protect	Physical page
2	1	0	0	18
17	1	0	0	2
6	1	0	0	4
12	1	0	0	12

Assume the following virtual addresses (represented in decimal) are accessed in the following order:

9, 72, 26, 17, 48, 74

For each access, state whether it is a TLB hit or TLB miss. Provide the corresponding physical page and physical address, if possible.

5.

You are given a data structure consisting of an array of unsigned dword values, starting at a memory location identified by the label `num_array`. After the last element in array, the end of the array is marked with a dword of value zero. You may assume that no element within the array is zero. That is:

```
num_array:
    dd 45
    dd 99
    dd 8190
    ; ....      some unknown number of array elements
    dd 0        ; end of array
```

Write an Intel assembly language program fragment that will (a) store the sum of the array in `eax` and (b) store the largest array value in `ebx`. Your solution must use a loop.

Hint: remember that in order to compare unsigned integers, you should use `cmp` followed by `jb` (jump if below), `ja` (jump if above), `jbe` (jump if below or equal), or `jae` (jump if above or equal).

6.

Write an E20 assembly language program that will store the value 1099 at memory cell 456, then halt. Your solution should consist of no more than seven instructions. Optional: you may use your E20 assembler concept (translating E20 assembly instructions to E20 Machine Codes manually following E20 manual) to make sure that your program is correct and can be assembled into valid machine code.